CEFET/RJ
DIRETORIA DE PESQUISA E PÓS-GRADUAÇÃO

UMA ANÁLISE DO USO DE LOTES DESLIZANTES NA DETECÇÃO DE EVENTOS EM SÉRIES TEMPORAIS EM STREAMING

Janio de Souza Lima

Proposta de Dissertação de Mestrado apresentada ao Programa de Pós-graduação em Ciência da Computação, Centro Federal de Educação Tecnológica Celso Suckow da Fonseca CEFET/RJ, como parte dos requisitos necessários à obtenção do grau de mestre.

Orientador: Eduardo Soares Ogasawara
Coorientadora: Rafaelli de Carvalho Coutinho

De acordo,

Eduardo Ogasawara
1803272

Rio de Janeiro,

Maio de 2022

RESUMO

Uma Análise do Uso de Lotes Deslizantes na Detecção de Eventos em Séries Temporais em Streaming

Janio de Souza Lima

Orientador: Eduardo Soares Ogasawara
Coorientadora: Rafaelli de Carvalho Coutinho

Resumo da Proposta de Dissertação submetida ao Programa de Pós-graduação em Ciência da Computação do Centro Federal de Educação Tecnológica Celso Suckow da Fonseca CEFET/RJ como parte dos requisitos necessários à obtenção do grau de mestre.

Detecção de eventos em séries temporais está relacionada ao estudo das técnicas para detecção de pontos em uma série com significado especial, que se diferenciam do comportamento esperado do conjunto de dados. A detecção de eventos tem aplicação em diversos cenários como identificação de fraudes, mudanças inesperadas em séries temporais financeiras, anomalias em dados médicos, segurança cibernética e monitoramento de sensores na exploração de petróleo. Em cenários de alta conectividade e onipresença da internet, marcados pela presença de gêmeos digitais e avanço de dispositivos IoT (internet das coisas), é possível observar o aumento na velocidade de geração dos dados em *streaming*. Diante disso, a identificação de eventos torna-se fundamental para a tempestividade na tomada de decisões, permitindo a correção e prevenção de situações indesejadas. Um aspecto relevante na detecção de eventos em séries em *streaming* é que apesar da detecção em lotes com a série completa (*offline*) não atender às necessidades de velocidade de resposta, a detecção instantânea (*online*) pode trazer custos computacionais que inviabilizem a aplicação prática de métodos. A fim de explorar a lacuna existente na literatura, o presente trabalho propõe uma análise do uso de lotes deslizantes, subsequências da série, na detecção de eventos em séries temporais em *streaming*, avaliando seus impactos na detecção precoce de eventos.

Palavras-chave:
Séries temporais, anomalias, eventos, *streaming*, *online*

Rio de Janeiro,

Maio de 2022

ABSTRACT

Uma Análise do Uso de Lotes Deslizantes na Detecção de Eventos em Séries Temporais em Streaming

Janio de Souza Lima

Advisors:
Eduardo Soares Ogasawara
Rafaelli de Carvalho Coutinho

Abstract of dissertation proposal submitted to Programa de Pós-graduação em Ciência da Computação - Centro Federal de Educação Tecnológica Celso Suckow da Fonseca CEFET/RJ as partial fulfillment of the requirements for the degree of master.

Detection of events in time series is related to the study of techniques for detecting points in a series with special meaning, which differ from the expected behavior of the data set. Event detection has application in several scenarios such as fraud identification, unexpected changes in financial time series, medical data anomalies, cyber security and sensor monitoring in oil exploration. In scenarios of high connectivity and ubiquity of the internet, marked by the presence of digital twins and the advancement of IoT devices (internet of things), it is possible to observe an increase in the speed of generation of *streaming* data. Therefore, the identification of events becomes essential for timely decision-making, allowing the correction and prevention of unwanted situations. A relevant aspect in detecting events in series in *streaming* is that although detection in batches with the full series (*offline*) does not meet the needs of speed of response, instant detection (*online*) can bring computational costs that make the practical application of methods unfeasible. In order to explore the gap in the literature, the present work proposes an analysis of the use of sliding batches, subsequences of the series, in the detection of events in time series in *streaming*, assessing their impacts on early detection of events..

Key-words:
Time series, anomaly, event, streaming, online

Rio de Janeiro,

Maio de 2022

Sumário

I Introdução **1**

II Referencial Teórico **4**

II.1 Séries Temporais 4

II.1.1 Estacionariedade de séries temporais 4

II.1.2 Mudança de conceito em séries temporais 6

II.2 Transformação de dados aplicada a séries temporais 7

II.3 Eventos em séries temporais 9

II.4 Séries temporais em *streaming* 11

III Trabalhos Relacionados **15**

III.1 Métodos de detecção de eventos em séries em *streaming* 15

III.2 Trabalhos de comparação de métodos 18

III.3 Síntese de Trabalhos Relacionados 20

IV Metodologia **23**

V Considerações Parciais **28**

Referências Bibliográficas 30

Lista de Figuras

II.1 Série temporal de queimadas em florestas no Brasil de 1998 a 2017 5

II.2 Exemplo de decomposição de séries 8

II.3 Taxonomia de detecção de eventos *online* 10

II.4 Matriz de Confusão 14

III.1 Evolução da quantidade de publicações 21

IV.1 Diagrama do *workflow* proposto 24

IV.2 Ilustração da detecção em lotes deslizantes 1 27

IV.3 Ilustração da detecção em lotes deslizantes 2 27

Lista de Tabelas

II.1 Métricas para avaliação de acertos das detecções 14

III.1 Trabalhos de comparação 19

Capítulo I Introdução

Em cenários de alta conectividade e onipresença da internet, marcados pela presença de gêmeos digitais, avanço de dispositivos IoT (internet das coisas) e tráfego de dados em plataformas de nuvem, é possível observar o aumento na velocidade de geração dos dados em *streaming* [Atzori et al., 2010; Habeeb et al., 2019]. A geração de dados em *streaming* se refere a dados sendo gerados em um fluxo contínuo em tempo real, por exemplo a partir do monitoramento de sensores. Diante disso, a identificação de situações indesejadas torna-se fundamental para a tempestividade na tomada de decisões, permitindo a correção e prevenção de situações indesejadas [Habeeb et al., 2019].

Quando os dados obtidos em *streaming* se referem a valores numéricos organizados sequencialmente de maneira cronológica, caracteriza-se uma série temporal. Dentre diversas tarefas realizadas em séries temporais, temos a detecção de eventos [Esling and Agon, 2012]. A detecção de eventos consiste na busca de padrões que não condizem com o esperado [Chandola et al., 2009], tendo aplicação em diversos cenários como identificação de fraudes, mudanças inesperadas em séries temporais financeiras, anomalias em dados médicos, segurança cibernética e monitoramento de sensores em equipamentos usados na exploração de petróleo.

Por outro lado, ao lidar com séries em *streaming*, a tarefa de detecção de eventos traz dificuldades adicionais. Por exemplo, mudança no comportamento dos dados ao longo do tempo e necessidade de adaptação à medida que novas observação dos dados em *streaming* são geradas [Talagala et al., 2020]. Além disso, aumenta a importância da velocidade de resposta dos métodos. Entretanto, apesar de haver uma miríade de métodos voltados para a detecção de eventos [Truong et al., 2020; Habeeb et al., 2019], ainda é possível observar uma escassez de trabalhos dedicados à experimentação abrangente de métodos de detecção de eventos voltados a dados em *streaming* com diferentes formas de aplicação dos métodos e diversidade dos conjuntos de dados usados como base das detecções [Ahmad et al., 2017; Lomio et al., 2020; Escobar et al., 2021; Wu and Keogh, 2021].

Na detecção de eventos *offline* há o acesso aos dados completos da série analisada, enquanto na *online* os métodos precisam analisar a série à medida que novas observações são geradas [Truong et al., 2020]. Assim, ainda que seja possível armazenar dados coletados de uma série temporal para sua posterior análise *offline* como um lote completo, esta tarefa contribui mais para entender fenômenos passados. Entretanto, para uso em tempo real das informações sobre eventos em séries

em *streaming*, faz-se necessária a detecção *online* [Rettig et al., 2015; Habeeb et al., 2019].

Apesar da detecção (*offline*) em lotes completos não atender às necessidades de velocidade de resposta, a detecção instantânea (a cada novo ponto na série) pode trazer custos computacionais que inviabilizem a aplicação prática de métodos [Talagala et al., 2020]. Neste contexto, deve-se notar que, ainda que o termo processamento em lote seja normalmente associado ao processamento *offline*, o processamento de dados em *streaming* pode ocorrer através do envio e recepção de dados individuais ou de pequenos lotes [Rettig et al., 2015; Hiraman et al., 2018]. Isso ocorre por exemplo com a ferramenta *Spark streaming* que usa o conceito de processamento micro lotes, ou seja, lotes pequenos dos dados para serem processados [Rettig et al., 2015]. A divisão da série em lotes foi experimentada, por exemplo, para o processamento em *streaming* para identificação de mudanças de conceito, uma tarefa que consiste na identificação de grandes mudanças em séries ao longo do tempo [Iwashita and Papa, 2019].

Ademais, a especificidade dos métodos para determinados comportamentos das séries e a necessidade de equilibro entre a detecção precoce de eventos e o custo do processamento *online* permitem levantar as seguintes hipóteses: (1) mudar o método aplicado quando a série muda de comportamento ao longo do tempo poderia levar a uma maior capacidade de adaptação das detecções; (2) a combinação de mais de um método levaria a resultados distintos na acurácia das detecções [Salles et al., 2020]; (3) o uso de lotes deslizantes que consigam lidar com subsequências menores da série poderia resultar em detecção precoce de eventos e redução do custo computacional do processamento *online*, mas sem os impactos negativos da detecção tardia do processamento *offline* [Talagala et al., 2020; Habeeb et al., 2019; Iwashita and Papa, 2019].

A fim de explorar a lacuna existente na literatura, o presente trabalho propõe uma análise do uso de lotes deslizantes na detecção de eventos em séries temporais em *streaming*. Desta forma, o trabalho contribui para testar diretamente a hipótese (3), proposta no parágrafo anterior, relacionada ao uso de lotes na detecção de eventos em séries em *streaming*. Indiretamente também há contribuição para exploração das duas primeiras hipóteses, através do aproveitamento dos resultados e experimentos a serem produzidos nesta pequisa, adaptando-os para experimentação e combinação de métodos de detecção de eventos em dados em *streaming* em trabalhos futuros.

Além disso, a realização de uma avaliação experimental mais abrangente durante a elaboração da dissertação contribuirá para: (i) avaliar o impacto na velocidade da capacidade de resposta aos eventos pela experimentação de diferentes configurações de lotes deslizantes em séries em *streaming* (tamanho de cada lote e memória dos dados); (ii) comparação de diferentes métodos de detecção *online* quanto à capacidade de detectar eventos corretamente e ao tempo de execução.

Além dessa introdução, o trabalho apresenta no Capítulo II o referencial teórico. No Capítulo III são analisados trabalhos relacionados à detecção de eventos em séries temporais. No Capítulo

IV é apresentada a metodologia para a análise do uso de lotes deslizantes na detecção de eventos em séries temporais em *streaming*, enquanto o Capítulo V tece considerações parciais sobre o trabalho.

Capítulo II Referencial Teórico

Este Capítulo apresenta um referencial teórico sobre séries temporais e detecção de eventos em séries temporais. A Seção II.1 discorre sobre os conceitos básicos sobre séries temporais e propriedades importantes deste tipo de conjunto de dados. A Seção II.2 trata outros aspectos das séries como transformação de dados aplicada a séries temporais, antes da aplicação de técnicas para sua análise. A Seção II.3 aborda os conceitos relacionados a eventos em séries temporais e sua detecção. Por fim, a Seção II.4 trata das séries temporais em *streaming* e seus impactos na detecção de eventos.

II.1 Séries Temporais

Uma série temporal é um conjunto de dados numéricos, organizados de forma sequencial e cronológica. A Equação II.1 representa uma série temporal, onde X é a série com n observações, x_1 é a observação mais antiga e x_n é a observação mais recente, ilustrada a seguir pela Figura II.1[1]. Esta representação refere-se a uma série com apenas uma variável, denominada série temporal **univariada**, enquanto uma série com diversas variáveis é denominada **multivariada** [Esling and Agon, 2012].

$$X = (x_1, ..., x_n), x_i \in R \tag{II.1}$$

Algumas características das séries temporais dificultam a aplicação de métodos estatísticos tradicionais ou requerem técnicas específicas de análises. Por exemplo, a autocorrelação entre as observações de uma série vai de encontro à assunção de independência e distribuição idêntica entre as observações adjacentes nos dados analisados feita por métodos estatísticos [Shumway and Stoffer, 2017]. Inclusive, é comum estudar o comportamento de uma série univariada em função de seus dados passados [Hanssens et al., 2003; Ogasawara et al., 2021].

II.1.1 Estacionariedade de séries temporais

A noção de estacionariedade refere-se a características de uma série que mantém regularidade de algumas medidas estatísticas, como média, variância e autocovariância, ao longo do tempo [Salles

[1] Dados disponíveis em: https://www.kaggle.com/gustavomodelli/forest-fires-in-brazil

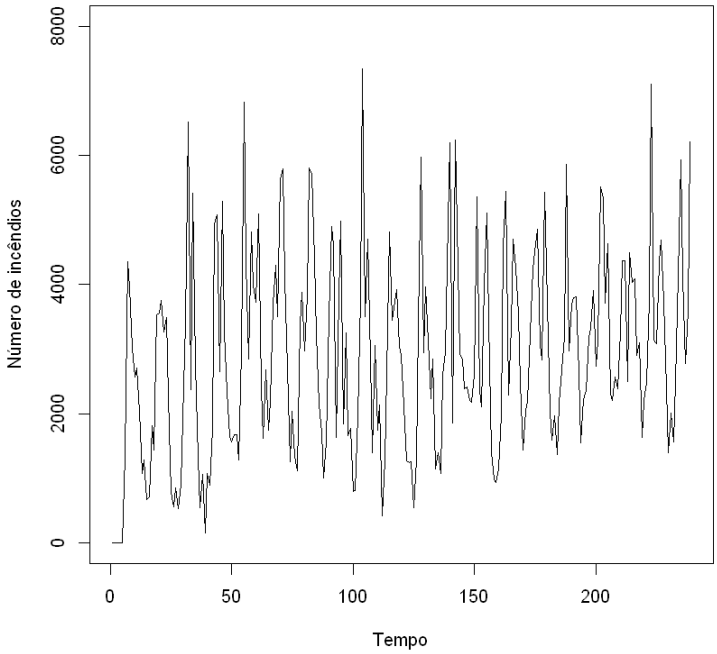

Figura II.1: Série temporal de queimadas em florestas no Brasil de 1998 a 2017: No eixo das ordenadas temos os valores das observações x_1 a x_n e no eixo das abscissas temos a representação do tempo no qual as observações estão distribuídas.

et al., 2019]. Muitos métodos aplicados à análise de séries temporais assumem sua estacionariedade. A forma mais básica, denominada **fracamente estacionária**, exige apenas que para uma subsequência x_t da série no tempo t: (i) a média μ_t e a variância σ_t^2 constantes e não dependentes do tempo t e (ii) para duas subsequências x_t e x_s, nos tempos s e t, a função de autocovariância dependa de s e t apenas através de sua diferença $|s-t|$ [Shumway and Stoffer, 2017].

Por outro lado, podemos dizer que uma série **estritamente estacionária** tem o comportamento probabilístico para quaisquer valores $(x_{t1}, x_{t2}, ..., x_{tk})$ idênticos ao conjunto deslocado no tempo $(x_{t1+h}, x_{t2+h}, ..., x_{tk+h})$, onde h representa o deslocamento no tempo. O comportamento probabilístico da série estritamente estacionária é demonstrado na Equação II.2, válida para todo $k = 1, 2, ...$, pontos no tempo $t_1, t_2, ..., t_k$, todos números $c_1, c_2, ...c_k$ e todos pontos deslocados no tempo $h = 0, \pm 1, \pm 2, ...$ [Shumway and Stoffer, 2017].

$$Pr\{x_{t1} \leq c_1, ..., x_{tk} \leq c_k\} = Pr\{x_{t1+h} \leq c_1, ..., x_{tk+h} \leq c_k\} \tag{II.2}$$

Por exemplo, se uma série for estritamente estacionária, para duas subsequências x_t e x_s, uma constante c e $k = 1$, temos $Pr\{x_t \leq c\} = Pr\{x_{s+h} \leq c\}$ para todos os pontos de s e t, onde s e t são intervalos de tempo diferentes. Contudo, é muito comum que uma série temporal seja não estacionária, pois violando qualquer das restrições de uma série estacionária, já podemos considerá-

la não estacionária. A ocorrência de não estacionariedade geralmente se dá pela mudança na média e variância da série, podendo indicar ocorrência de eventos [Ogasawara et al., 2021].

Como efeito da não estacionariedade, o comportamento típico dos dados muda à medida que novas observações ocorrem com pontos de mudança ou alterações na sazonalidade e ciclos. Desta forma, modelos aprendidos baseados em observações passadas perdem a capacidade de explicação do comportamento futuro da série. Para lidar com este problema, os modelos na análise de séries não estacionárias precisam ser atualizados periodicamente [Talagala et al., 2020].

II.1.2 Mudança de conceito em séries temporais

A mudança de conceito em séries temporais tem relação com comportamento não estacionário e ocorre quando alterações no processo gerador da série mudam seu comportamento [Iwashita and Papa, 2019; Demšar and Bosnić, 2018; Žliobaitė, 2010]. A existência de mudança de conceito pode tornar inadequadas tarefas dependentes de modelos aprendidos da série antes da mudança de conceito [Iwashita and Papa, 2019; Žliobaitė, 2010]. Por isso, há diversas técnicas para lidar com aprendizado de padrões que mudam ao longo do tempo para identificar as mudanças de conceitos, podendo ser divididas em: (i) baseadas em janelas, (ii) baseadas em pesos e (iii) *ensemble* de classificadores [Iwashita and Papa, 2019].

Na abordagem de janelas ou seleção de instâncias é usado um conjunto adequado de dados anteriores para treinar classificadores. São selecionadas instâncias de dados dentro de uma janela deslizante da série temporal, esquecendo instâncias antigas à medida que o treinamento avança, por considerá-las sem uso para o aprendizado de mudança de conceito. Assim, o foco do treinamento é o último lote de informações da série analisada [Iwashita and Papa, 2019].

A identificação de mudanças de conceito baseada em pesos atribui peso às instâncias da série e esquece os pesos de instâncias antigas. Assim, à medida que o tempo decorre o peso de dados antigos vai sendo reduzido na identificação das mudanças. Nesta abordagem deve ser usado um classificador que possa usar os pesos para aprendizado e identificação das mudanças de conceito [Iwashita and Papa, 2019].

Por sua vez, a abordagem baseada em *ensemble* de classificadores combina os resultados de um conjunto de classificadores de aprendizado de conceito. A combinação pode usar tanto votos dos diferentes classificadores quanto a seleção dos resultados mais significantes [Iwashita and Papa, 2019]. O uso de *ensemble* em tarefas de mineração em geral parte da intuição que os resultados combinados de diversos métodos serão mais acurados que os resultados isolados de cada método [Han et al., 2011].

II.2 Transformação de dados aplicada a séries temporais

Para lidar com características que dificultam a aplicação de técnicas tradicionais em séries temporais, por exemplo a não estacionariedade, podem ser realizadas tarefas de transformação, gerando versões ou representações da série temporal. Como parte das tarefas de pré-processamento de dados, há diversas técnicas para transformação de séries temporais como decomposição, normalização, suavização, extração de janelas deslizantes e agrupamento de subsequências [McLaren et al., 2018; Ogasawara et al., 2010; Shumway and Stoffer, 2017; Lin et al., 2003; Chiu et al., 2003]. A seguir são apresentadas algumas destas técnicas que fazem parte da preparação de dados para melhorar a qualidade dos dados antes da aplicação de tarefas de mineração mais complexas [Ogasawara et al., 2010; Han et al., 2011].

A decomposição de séries temporais visa identificar componentes latentes da série que normalmente não são observáveis. Os principais componentes de uma série são: tendência, ciclos, sazonalidade e resíduos. A tendência indica a direção de subida ou queda dos valores das observações. Os ciclos são ocorrências de picos ou vales de maneira repetitiva ao longo do tempo. A sazonalidade é similar aos ciclos, mas refere-se a sua ocorrência repetitiva em cada ano. Os resíduos são os componentes restantes, relativos a variações não previsíveis [McLaren et al., 2018].

Para ilustrar o que são os componentes e a decomposição de séries temporais, pode-se analisar a Figura II.2, que retoma a série apresentada na Figura II.1 e aplica a decomposição. O primeiro gráfico da Figura II.2 representa a série original com dados de queimadas no Brasil no período de 1998 a 2017[2]. Os demais gráficos representam, respectivamente, os componentes latentes presentes na série: tendência, sazonalidade e resíduos ou ruído aleatório, que não podem ser observados diretamente na série original.

A decomposição apresentada na Figura II.2 é realizada removendo a tendência, através da Equação II.3, onde X_t é a série original e T_t é o componente de tendência. Em seguida é removida a sazonalidade, através da Equação II.4, onde S_t é o componente sazonal. Com isso, chega-se à série $\hat{X}_t$ apenas com resíduos ou ruído aleatório [Escobar, 2020].

$$\hat{X}_t = X_t - T_t \tag{II.3}$$

$$\hat{X}_t = \hat{X}_t - S_t \tag{II.4}$$

A decomposição de uma série permite entender melhor o seu comportamento pelo estudo dos componentes latentes, mesmo em uma análise exploratória inicial. Outra aplicação comum da decomposição é a aplicação de técnicas de detecção de eventos na série diferenciada (série obtida

[2]Dados disponíveis em: https://www.kaggle.com/gustavomodelli/forest-fires-in-brazil

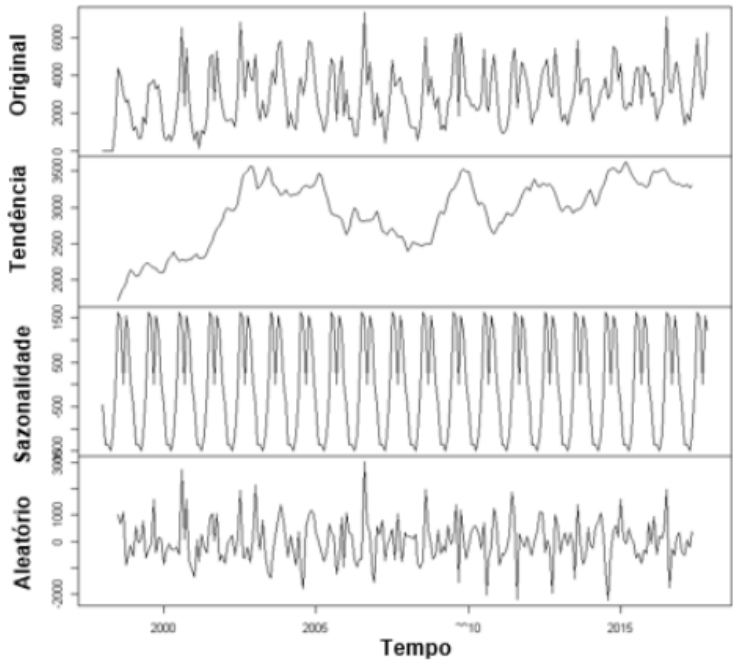

Figura II.2: Exemplo de decomposição de séries: Série temporal de queimadas em florestas no Brasil 1998 a 2017 decomposta em tendência, sazonalidade e componente aleatório.

pela remoção de um ou mais componentes), lidando com a não estacionariedade relacionada a algum dos componentes. Um problema de técnicas que usam a decomposição é o alto custo de processamento [Marik and Bohac, 2019].

Outras técnicas relevantes de transformação de dados em séries temporais são a normalização e suavização de séries temporais que, assim como na transformação de dados em geral [Han et al., 2011], geram representações simbólicas da série que podem tornar as tarefas de análise mais eficientes. Como forma de suavização, é comum o uso de médias móveis. As médias móveis, muito usadas em finanças e econometria, reduzem o efeito de ruídos e auxiliam na identificação de mudanças no comportamento da série [Ogasawara et al., 2010].

A normalização de dados consiste em mapear valores equivalentes para cada observação do conjunto de dados em uma escala diferente, por exemplo dentro de um intervalo definido [Han et al., 2011]. Estas representações auxiliam no equilíbrio do peso entre atributos diferentes em um conjunto de dados e facilita tarefas como agrupamento e classificação. Os métodos *min-max*, *z-score* e escala decimal estão entre os mais comuns para normalização [Ogasawara et al., 2010; Han et al., 2011].

O método *min-max* gera uma representação em que cada observação estará em um intervalo definido de valores mínimo e máximo, mas se os valores da séries não forem conhecidos a definição destes limites pode gerar erros pela concentração dos dados normalizados em uma faixa indevida. A normalização pelo *z-score* usa os valores de média e desvio padrão dos dados, portanto pode ser afetada por séries não estacionárias nas quais estas medidas variam ao longo do tempo. Por sua vez,

a normalização pela escala decimal consiste no deslocamento do ponto decimal dos valores da série de acordo com o valor máximo existente [Han et al., 2011]. Outra técnica que pode ser aplicada nesta tarefa é a normalização adaptativa [Ogasawara et al., 2010], que consiste em transformar a série não estacionária em estacionária usando a técnica de janelas deslizantes, remover valores discrepantes e somente depois destes passos normalizar a série.

Para analisar propriedades locais em uma série X, podem ser extraídas subsequências de tamanho p, formalmente definidas na Equação II.5. As subsequências extraídas são valores contínuos na série no intervalo definido na Equação II.5 e podem ser usadas tanto como tarefa de pré-processamento como para busca de padrões, técnica de análise de tendência ou discretização de dados [Fu, 2011; Chiu et al., 2003]. Uma técnica comum de subsequências é a extração de janelas deslizantes.

$$seq_{p,i}(X) =< X_i, X_{i+1}, ..., x_{i+(p-1)} >, |seq_{p,i}(X)| = p, 1 \leq i \leq |X| - p \tag{II.5}$$

A técnica de janelas deslizantes consiste em extrair subsequências que percorrem toda a série, gerando um conjunto de subsequências de mesmo tamanho, podendo ser representada como uma matriz A contendo todas as subsequências da série X, como definida na Equação II.6. A extração de janelas deslizantes é útil para trabalhar com análises das propriedades locais de cada janela, mas ao mesmo tempo analisar uma representação da série original completa, deslizando através de suas subsequências ou analisando cada janela como um termo defasado da série [Lin and Keogh, 2004; Ogasawara et al., 2010]. Também utiliza-se as janelas deslizantes para realizar o agrupamento de subsequências, tarefa que trata cada janela como uma série individual e busca mensurar as similaridades entre elas para identificar grupo de janelas semelhantes, mesmo que não sejam contínuas [Lin et al., 2003; Chiu et al., 2003].

$$sw_p(X) = A, \forall a_i \in A, a_i = seq_{p,i}(X) \tag{II.6}$$

II.3 Eventos em séries temporais

A mineração de dados em séries temporais busca identificar informações ocultas nos dados analisados e engloba tarefas como consulta por conteúdo, agrupamento, classificação, segmentação, predição, detecção de eventos e descoberta de *motif* [Esling and Agon, 2012; Fu, 2011]. Estas tarefas representam uma organização didática dos temas, mas podem ocorrer de maneira conjunta e normalmente um tipo de tarefa usa resultados de outra ou lhe é complementar [Esling and Agon, 2012]. Nesta Seção serão discutidos conceitos relacionados à tarefa de detecção de eventos.

Ao pesquisar por eventos em séries temporais, alguns termos são encontrados na literatura de

maneira intercambiável como eventos, anomalias, discrepâncias, valores atípicos. Alguns autores como Chandola et al. [2009] simplesmente consideram alguns deles como sinônimos, como é o caso de anomalias e valores atípicos. Por outro lado, às vezes eventos e anomalias são tratados como subcategorias um do outro, como citado por Marik and Bohac [2019] que referem-se a anomalias como algo mais genéricos e eventos como um padrão mais complexo encontrado na série.

Neste trabalho será usado o termo evento de forma mais geral para aludir a pontos em uma série temporal com significado especial. Apesar de identificados como pontos específicos, tem relação com intervalos da série temporal ao seu redor. O eventos podem referir-se por exemplo a anomalias e pontos de mudança [Gensler and Sick, 2018].

A detecção de eventos refere-se à detecção automática de pontos com significado especial em uma série temporal. Uma forma de dividir a análise das séries para detecção de eventos é quanto ao modo de acesso aos dados em *online* e *offline*. Quando o acesso é em tempo real, à medida que são geradas novas observações, a análise é denominada detecção *online* e denomina-se detecção *offline* quando se tem acesso prévio ao conjunto de dados completo [Truong et al., 2020].

A Figura II.3 apresenta uma taxonomia para detecção *online* de eventos em séries temporais, proposta por [Ogasawara et al., 2021] baseada numa análise da literatura sobre o tema e inspirada em [Habeeb et al., 2019]. A taxonomia proposta divide o estudo de detecção *online* de eventos em séries nas categorias: (i) tipos e (ii) granularidade de eventos, (iii) forma de aprendizado, (iv) estratégia, (v) abordagem e (vi) métodos para detecção de eventos e (vii) tarefas relacionadas ao tratamento de séries no cenário de *Big Data* [Ogasawara et al., 2021]. O restante desta Seção trata sobre tipos de eventos, granularidade, forma de aprendizado e estratégias, alguns dos aspectos relevantes desta taxonomia e da literatura em geral para o estudo de eventos em séries temporais.

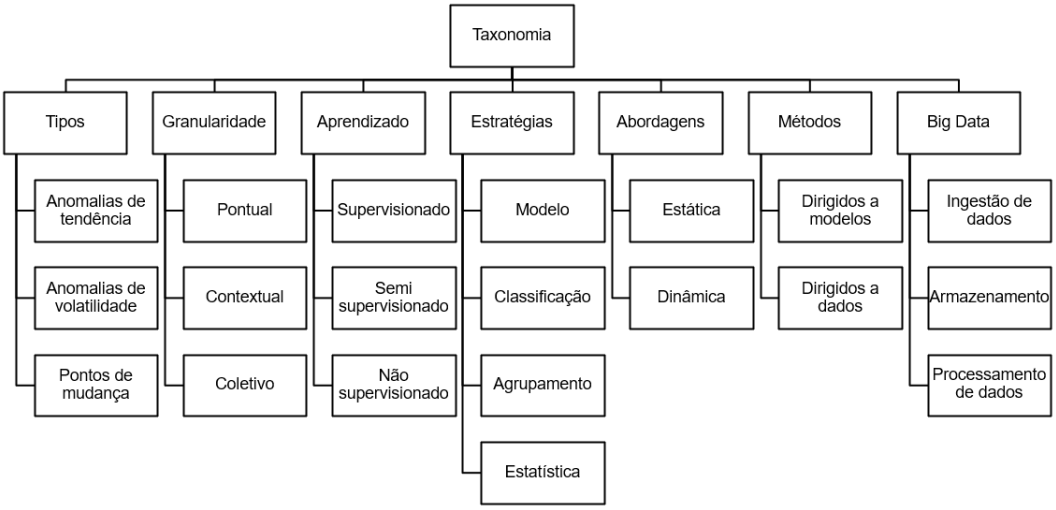

Figura II.3: Taxonomia de detecção de eventos *online*. Adaptada de Ogasawara et al. [2021]

Um tipo comum de eventos são as anomalias, que se referem a observações que não condizem com o comportamento esperado do conjunto de dados [Chandola et al., 2009]. Quanto aos tipos, as

anomalias podem ser de tendência ou de volatilidade. As anomalias de tendência ocorrem quando um evento escapa da tendência esperada da série, enquanto as anomalias de volatilidade ocorrem quando há uma grande mudança na volatilidade da série analisada [Ogasawara et al., 2021].

Pontos de mudança são eventos que indicam uma mudança repentina na série temporal. Sua identificação é relevante porque após o ponto de mudança é raro identificar a sequência de observações em uma série temporal com base em seu comportamento antes da mudança. O principal interesse na identificação de pontos de mudança é em mudanças abruptas, pois mudanças mais suaves são naturais nas séries [Takeuchi and Yamanishi, 2006].

Conforme ilustrado pela segunda categoria da Figura II.3, quanto a sua granularidade, os eventos podem ser divididos em pontuais, coletivos ou contextuais [Chandola et al., 2009; Han et al., 2011]. Os pontuais ocorrem quando um ponto individual já é considerado anômalo em relação aos demais dados. Os coletivos ocorrem quando um ponto individualmente não é considerado um evento, mas sua ocorrência em conjunto com outros pontos não condiz com o comportamento do conjunto de dados. Quando a discrepância é condicionada a determinado contexto, relacionado à estrutura do conjunto de dados, temos um evento contextual [Chandola et al., 2009; Han et al., 2011].

Dependendo da disponibilidade de rótulos para identificar a ocorrência de eventos as técnicas de detecção podem ser classificadas como supervisionada, semi supervisionada ou não supervisionada [Chandola et al., 2009; Habeeb et al., 2019; Han et al., 2011]. As técnicas supervisionadas ocorrem quando há disponibilidade de rótulos para treinamento tanto para os casos normais quanto para anomalias, enquanto quando há apenas rótulos para os casos normais tem-se técnicas semi supervisionadas. Quando não há qualquer rótulo tem-se a detecção não supervisionada.

Outra importante classificação das técnicas de detecção de eventos é quanto à estratégia empregada, podendo ser baseada em classificação, baseada em agrupamento, análise de desvio de modelos ou técnicas estatísticas [Chandola et al., 2009]. As técnicas baseadas em classificação e agrupamento, tratam a detecção como tarefas de classificação e agrupamento, respectivamente, onde se busca atribuir uma classe ou grupo para cada ponto na série. A análise de desvio de modelo, consiste no treinamento prévio de um modelo e posterior comparação das novas observações para identificar como eventos aquelas que desviam do modelo treinado. Por sua vez, técnicas estatísticas trabalham com a análise da distribuição dos dados e marcação de eventos para observações que não estão em conformidade com a distribuição identificada para a série [Ogasawara et al., 2021].

II.4 Séries temporais em *streaming*

O *streaming* de dados está relacionado ao tráfego e processamento de dados em tempo real, antes de seu armazenamento. Este tipo de processamento de dados é muito comum em diferentes cenários

como redes sociais, monitoramento de sensores, plataformas de *streaming* de vídeo e monitoramento de séries financeiras, como preço de ações. Uma forma comum para o processamento de dados em *streaming* é conhecida como sistemas de mensageria, em que os dados trafegam como mensagens [Hiraman et al., 2018].

Há dois padrões principais para sistemas de dados em *streaming*: (i) ponto a ponto (P2P) e (ii) publicar/subcrever, também conhecido como *pub/sub*. No padrão P2P as aplicações conectam-se entre si, enviando e recebendo dados trafegados em uma fila. No segundo padrão há publicadores de conteúdo e consumidores deste conteúdo, sem necessidade de conexão direta entre as aplicações [Hiraman et al., 2018].

No contexto de processamento de dados em *streaming* é importante citar a existência de plataformas especializadas nesta tarefa. Habeeb et al. [2019] citam *Apache Kafka* e *Spark streaming* como exemplos de plataformas específicas para processamento de *streaming* de dados. Estas plataformas, permitem troca de dados em tempo real e tratamento de grandes volumes de dados [Hiraman et al., 2018].

Além do tráfego de dados, as plataformas de *streaming* apresentam diversas possibilidade de análise destes dados na forma de séries temporais, tais como preços de ações, tráfego de rede, consumo de recursos da nuvem, vazamentos em dutos de petróleo. Em resumo, qualquer dado de natureza numérica trafegando em *streaming* que possa ser organizado de maneira cronológica e regular pode ser analisado como uma série temporal em *streaming*. Há diversas tarefas na análise destas séries temporais, como agregações, predições, detecção de fraudes e detecção de eventos [Carter and Streilein, 2012].

Na detecção de eventos *offline* os métodos precisam do acesso aos dados completos da série analisada, enquanto na *online* os métodos analisam a série à medida que novas observações são geradas [Truong et al., 2020]. Assim, ainda que seja possível armazenar dados coletados de uma série temporal para sua posterior análise *offline* como um lote completo, esta tarefa contribui mais para entender fenômenos passados. Entretanto, ao lidar com eventos em séries em *streaming* o foco é no uso das informações em tempo real ou próximo ao tempo real, por isso se faz necessária a detecção *online* [Rettig et al., 2015; Habeeb et al., 2019].

Apesar da detecção em lotes completos (*offline*) não atender às necessidades de velocidade de resposta, a detecção a cada novo ponto na série pode trazer custos computacionais que reduzem a aplicação prática de métodos [Talagala et al., 2020]. Neste contexto, deve-se notar que, ainda que o termo processamento em lote seja normalmente associado ao processamento *offline*, mesmo o processamento de dados em *streaming* pode ocorrer através do envio e recepção de dados individuais ou de pequenos lotes [Truong et al., 2020; Rettig et al., 2015; Hiraman et al., 2018]. Isso ocorre por exemplo com a ferramenta *Spark streaming* que usa o conceito de processamento de micro lotes,

ou seja, lotes pequenos dos dados para serem processados [Rettig et al., 2015]. A divisão da série em lotes foi experimentada por exemplo para o processamento em *streaming* para identificação de mudanças de conceito [Iwashita and Papa, 2019].

A detecção *online* de eventos de série temporais em *streaming*, apesar de relevante para monitoramento dos dados, traz novos desafios como ausência de rótulos identificando os eventos reais, capacidade de adaptação para diferentes tipos de séries e aprendizado de parâmetros com os próprios dados [Carter and Streilein, 2012]. Além disso, os dados nas séries em *streaming* podem passar a maior parte do tempo com comportamento normal, gerando conjuntos de dados desbalanceados quanto às classes normais e anômalas. Por isso o monitoramento e a identificação de eventos podem ter alto consumo de recursos e ser ineficientes.

Ademais, o comportamento de série em *streaming*, como mudanças ao longo do tempo, presença de não estacionariedade, dificulta a aplicação de abordagens tradicionais usadas na detecção *offline* de eventos [Talagala et al., 2020; Ahmad et al., 2017]. O trabalho com lotes completos de dados, por exemplo, é incompatível com a necessidade de detecção precoce de eventos [Talagala et al., 2020]. Sem a detecção precoce em tempo real (a cada nova observação) ou próxima ao tempo real, a capacidade de resposta aos eventos fica prejudicada.

Na literatura há duas abordagens principais para lidar com a detecção *online* de eventos em séries em *streaming*: modelos estáticos e modelos dinâmicos. Nos modelos estáticos o método treina um modelo em grandes volumes de dados e posteriormente o aplica no *streaming* de dados, marcando cada observação como anomalia ou não à medida que há desvios em relação ao modelo treinado. No segundo, o treinamento é feito apenas como aquecimento para iniciar o modelo e realizar as detecções, mas à medida que novos dados são recebidos via *streaming*, o modelo é retreinado continuamente de maneira incremental [Habeeb et al., 2019].

A Tabela II.1 apresenta métricas comuns em tarefas de classificação como acurácia, precisão, revocação e F_1 [Han et al., 2011]. Comumente, estas métricas são usadas ao analisar o desempenho de métodos de detecção de eventos [Ren et al., 2019; Salles et al., 2020; Hasani, 2020]. O resultado da tarefa de detecção de eventos normalmente contém um vetor lógico (verdadeiro/falso) ou binário (1/0) que indica se cada observação da série é um evento. A partir da comparação dos resultados obtidos com os rótulos indicando os eventos reais, é possível gerar uma matriz de confusão.

A Figura II.4 ilustra um exemplo de uma matriz de confusão, na qual são distribuídos os valores da contagem de observações da série com sua classificação. Os valores da matriz de confusão indicam a contagem de: verdadeiros positivos (VP), acertos de existência de eventos; verdadeiros negativos (VN), acertos de inexistência de eventos; falsos positivos (FP) e falsos negativos (FN) [Han et al., 2011]. Estes valores são a base para o cálculo das principais métricas descritas na Tabela II.1.

A matriz de confusão pode ser usada em classificações mais complexas em outros tipos de

		Eventos Detectados	
		Sim	Não
Eventos Reais	Sim	VP	FP
	Não	FN	VN

Figura II.4: Matriz de Confusão. Adaptada de Han et al. [2011]

conjuntos de dados com várias classes, mas a matriz adaptada na Figura II.4 é o tipo mais básico, usada normalmente para classificações em conjuntos de dados de duas classes [Han et al., 2011]. Por isso, este tipo de matriz é adequada para análises dos eventos, pois facilita a análise simplificada de acertos (VP, VN) e erros (FP, FN). Sua interpretação é intuitiva, pois visualmente é possível verificar o desempenho de um método que será melhor à medida que houver maior concentração de observações na diagonal formada pelos acertos.

Tabela II.1: Métricas para avaliação de acertos das detecções: obtidas a partir da matriz de confusão

Métrica	**Descrição**	**Fórmula**
Acurácia (AC)	Proporção de acertos em relação ao total de observações	$\frac{VP+VN}{VP+VN+FP+FN}$
Precisão (PR)	Medida de exatidão	$\frac{VP}{VP+FP}$
Revocação (RV)	Medida de completude ou taxa de verdadeiro positivo	$\frac{VP}{VP+FN}$
F_1	Medida que avalia o equilíbrio entre precisão e revocação	$2 \times \frac{PR \times RV}{PR+RV}$

As métricas apresentadas na Tabela II.1, por sua vez, permitem uma análise mais aprofundada que a matriz de confusão. Com a análise da acurácia é possível ter uma visão geral dos acertos, ou seja, indica se determinado método consegue acertar tanto a existência de eventos quanto a inexistência. A precisão fornece informações sobre o nível de acertos dos eventos existentes, mas não consegue identificar se foram marcados muitos pontos como falsos negativos. De maneira inversa, a revocação consegue avaliar se o método classificou o maior número de eventos possíveis, mas não captura excesso de falsos positivos. Como a precisão e revocação tendem a ter uma relação inversamente proporcional, a métrica F_1 é obtida pelo calculo da médica harmônica entre precisão e revocação, permitindo avaliar se um método obteve resultados equilibrados [Han et al., 2011].

Capítulo III Trabalhos Relacionados

Para exploração da área de pesquisa sobre detecção de eventos *online* em séries temporais inicialmente foram realizadas buscas de artigos indexados na base *Scopus*. Para seleção foi usada a *string* de busca *"time series"AND ("anomaly detection"OR "event detection") AND ("online"OR "streaming")* aplicada aos títulos, resumos e palavras chave, obtendo-se 379 documentos na última consulta em março de 2022. Para resultados relevantes foram explorados os documentos, eliminando-se artigos sem citações, exceto quando publicados nos últimos 12 meses. Foram analisados os resumos dos documentos mantidos no filtro para identificar associação ao tema estudado. Além disso, para expandir a pesquisa foi aplicada a técnica de *snowballing* a partir dos artigos lidos. Os trabalhos relacionados adiante retratam o estágio atual dessa exploração que deve ser continuada e refinada durante o desenvolvimento da dissertação.

A discussão dos trabalhos relacionados foi organizada neste Capítulo para explorar tanto as especificidades dos métodos quanto as formas de comparação destes métodos. Para isso, os métodos de detecção de eventos são explorados na Seção III.1 e trabalhos de comparação, métricas e conjuntos de dados são discutidos na Seção III.2. O Capítulo é concluído com a Seção III.3 que sintetiza os achados da análise dos trabalhos, aborda lacunas identificadas e aspectos relevantes para a evolução da pesquisa.

III.1 Métodos de detecção de eventos em séries em *streaming*

Na literatura há diversos métodos aplicáveis à detecção de eventos *online*. Uma visão geral sobre o estado da arte nesta área é apresentada por Habeeb et al. [2019], cujo estudo apresenta duas principais divisões na detecção de eventos *online*: (i) modelos estáticos e (ii) modelos de aprendizado dinâmico. Modelos estáticos são treinados em grandes conjuntos de dados e aplicados em dados em tempo real, enquanto nos modelos de aprendizado dinâmico o treinamento é iniciado em um subconjunto de dados, como uma espécie de aquecimento, e a aprendizagem continua à medida que novas observações são recebidas [Habeeb et al., 2019; Ogasawara et al., 2021].

Há diversos métodos desenvolvidos para detecção *offline*, contudo trabalhos como de Talagala et al. [2020]; Habeeb et al. [2019] indicam dificuldade destes métodos lidarem com séries em *streaming*. Deste modo, esta seção foca em métodos voltados para detecção em tempo real. Contudo,

segundo o melhor conhecimento dos autores, não há evidências definitivas da inaplicabilidade de métodos indicados inicialmente para detecção *offline* no cenário de *streaming*. Diante disto e tendo em vista seu uso amplamente difundido para tipos de séries ou de eventos específicos que não devem ser desprezados [Salles et al., 2020; Truong et al., 2020; Salles et al., 2019; Chandola et al., 2009], é importante também citá-los nesta seção.

Alguns exemplos de métodos desenvolvidos inicialmente para detecção *offline* de acordo com o tipo de eventos são: (i) **Anomalias de volatilidade**: modelos econométricos como *Generalized Autoregressive Conditional Heteroskedasticity* (GARCH) que estima a volatilidade baseada nas observações passadas da série [Carmona, 2014]; (ii) **Anomalias de tendência**: métodos baseados em modelos para identificar a tendência da série como *Holt-Winters* (HW) [Hasani, 2017] ou decomposição como *Anomalize* [Dancho and Vaughan, 2020]; (iii) **Pontos de mudança**: métodos baseado em suaviazação como *Exponentially Weighted Moving Average* (EWMA) ou especializados na detecção de mudanças como *Seminal Change Point* (SCP) e *ChangeFinder* (CF) [Guralnik and Srivastava, 1999; Takeuchi and Yamanishi, 2006; Escobar et al., 2021].

Com foco na detecção *online* de eventos, um método é proposto por Rettig et al. [2015] visando escalabilidade e generalização e usando as ferramentas de processamento de *streaming Apache Kafka* e *Spark streaming*. O método usa as métricas entropia relativa e correlação de Pearson para avaliar os dados e identificar anomalias, sem assumir previamente uma distribuição para os dados. A combinação destas métricas permite identificar: (i) mudanças graduais no *stream* de dados com a entropia relativa e (ii) mudanças abruptas com a correlação de Pearson [Rettig et al., 2015].

Ahmad et al. [2017] propõem um método para detecção de eventos em séries temporais em *stream*. O método é baseado no algoritmo Memória Temporal Hierárquica (MTH), um *framework* teórico para aprendizado de sequências de neurônios corticais. O MTH usa aprendizado de máquina para modelar a distribuição da série e comparar novas observações com as predições do modelo, marcando eventos de acordo com a divergência entre os valores reais e preditos [Ahmad et al., 2017; Hasani, 2017].

O método *k-Nearest Neighbors-Conformal Anomaly Detector* (k-NN-CAD) [Ishimtsev et al., 2017] identifica eventos em séries univariadas de maneira *offline* e *online*. O k-NN-CAD combina mensuração de distância com análise de fronteiras, para isso usa o algoritmo k-vizinhos próximos (k-NN) e os algoritmos *conformal anomaly detection* (CAD) e *inductive conformal anomaly detection* (ICAD) [Laxhammar, 2014]. O k-NN-CAD realiza a detecção em duas etapas principais: (i) cria uma representação da série usando a técnica de janelas deslizantes [Lin et al., 2003; Chiu et al., 2003] e (ii) aplica nesta representação uma análise baseada em distância com o algoritmo k-NN para mensurar se as observações analisadas estão em conformidade com um limite (fronteira) da distância esperada para os demais dados da série [Ishimtsev et al., 2017].

O método *FuseAD* [Munir et al., 2019] busca detectar eventos em diferentes tipos de séries temporais aplicando aprendizado não supervisionado. Para tanto, o FuseAD funciona através da fusão de métodos estatísticos e aprendizado de máquina, usando o modelo auto-regressivo integrado de médias móveis (ARIMA) [Shumway and Stoffer, 2017] e uma rede neural convolucional (CNN) na etapa de predição e um detector de anomalias na etapa final. A combinação do ARIMA e CNN é usada para predição dos próximos valores e as novas observações da série são marcadas como eventos em função de sua distância em relação aos valores preditos [Munir et al., 2019].

Um método proposto por Ren et al. [2019] foi implementado como um serviço denominado *Microsoft Anomaly Detector* (MAD)[1] na plataforma de nuvem *Microsoft Azure*. O MAD usa o algoritmo SR-CNN, uma combinação do algoritmo *spectral residual* (SR) e uma CNN [Ren et al., 2019]. O SR realiza uma transformação da série gerando uma representação denominada mapa de saliência na qual os valores extremos são acentuados, enquanto a CNN usa esta versão transformada da série para aprender as fronteiras para determinar a ocorrência de eventos. Outro serviço de detecção de eventos em uma plataforma de nuvem é o *Amazon QuickSight ML-powered anomaly detection*[2] da plataforma AWS, contudo não foram identificadas publicações com detalhes sobre seu funcionamento ou o método usado por este serviço para comparação neste trabalho.

Um *framework* para detecção *online* de eventos em séries temporais multivariadas é proposto por Talagala et al. [2020]. Este *framework* é baseado na teoria de valores extremos (EVT) para o cálculo de fronteiras do comportamento normal dos dados para identificar mudanças significativas em novas observações. A detecção de eventos proposta lida com a não estacionariedade através da divisão da série em janelas deslizantes e ajustando o modelo à medida que mudanças no comportamento dos dados são detectadas [Talagala et al., 2020].

Outro método que emprega uma etapa de predição e outra de detecção baseada na divergência entre observações reais do *streaming* e as predições do modelo é o filtro de mediana - memória de curto prazo longa empilhada - média móvel exponencialmente ponderada (MF-*stacked* LSTM-EWMA) [Zhang et al., 2020]. Na verdade o método MF-*stacked* LSTM-EWMA é dividido em três etapas: (i) uso do algoritmo MF para identificar anomalias óbvias, (ii) *stacked* LSTM como preditor das próximas observações da série e (iii) EWMA para detectar eventos, criando uma fronteira com limites para as divergência entre os valors preditos e as observações reais. Na etapa de predição ao invés de uma única LSTM, é usada uma abordagem em que são empilhadas camadas de LSTM para obter as predições que serão usadas como base para detecção dos eventos [Zhang et al., 2020].

Ao avaliar os trabalhos relacionados, citados nesta Seção, percebe-se que tem relação com diversos aspectos da taxonomia apresentada na Seção II.3 do Capítulo II. Contudo, é possível classificá-los de maneira simplificada quanto à forma de detectar um evento em: (i) detecção baseada

[1]Disponível em: https://azure.microsoft.com/en-us/services/cognitive-services/anomaly-detector/
[2]Disponível em: https://docs.aws.amazon.com/quicksight/latest/user/anomaly-detection.html

em divergência de **valores preditos** e (ii) detecção baseada em **fronteiras**.

Na detecção baseada em valores preditos os métodos possuem uma etapa prévia em que é realizada a predição dos próximos valores da série temporal e cada nova observação recebida no *streaming* é comparada com a predição, sendo marcados como eventos quando há divergência entre o real e o predito. A detecção baseada em fronteiras, por sua vez, consiste em definir limites inferiores e superiores além dos quais cada nova observação recebida no *streaming* é marcada como um evento. Os métodos MTH, FuseAD e MF-*stacked* LSTM-EWMA, citados anteriormente, poderiam ser classificados como detecção baseada em valores preditos, enquanto que k-NN-CAD, MAD e EVT, em detecção baseada em fronteiras.

III.2 Trabalhos de comparação de métodos

Além de trabalhos focados em propor métodos para detecção de eventos, buscou-se identificar estudos de comparação de métodos. Este refinamento da busca inicial da seleção de trabalhos relacionados foi realizado para contribuir no entendimento desta área de pesquisa e das metodologias adotadas para executar experimentos de comparação de métodos. Para isso foi realizado filtro e *snowballing* procurando por trabalhos com títulos contendo os termos *survey*, *benchmark* e *comparison* nos 379 documentos da *string* de busca original. Esta seção apresenta estes trabalhos, com uma breve descrição de como foram realizadas as comparações, métricas e conjuntos de dados usados e métodos melhor avaliados em cada comparação.

O *Harbinger* é um *framework* que disponibiliza funções para detecção de eventos, avaliação, visualização, combinação de métodos e comparação das detecções realizadas [Salles et al., 2020]. Ao invés de um método específico, o *Harbinger* possibilita a implementação e a combinação de diferentes métodos de detecção de eventos, mas os executa de maneira *offline*. A pesquisa analisou sete métodos através de suas implementações no *Harbinger*, usando a métrica F_1 para comparar o desempenho das detecções, tendo apontado resultados superiores para os métodos *Adaptive Normalization* (AN), decomposição, k-NN-CAD e EWMA [Salles et al., 2020].

Uma análise de métodos para detecção *online* de eventos e uma comparação do desempenho, tempo de execução e uso de CPU, são apresentadas por Hasani [2020]. Foram analisados os métodos ARIMA, médias móveis, Holter Winter (HW) e HW-GA [Hasani, 2020]. Para cálculo das métricas foi realizado monitoramento em tempo real do uso de CPU e tempo de execução de cada método [Hasani, 2020].

A pesquisa de Ahmad et al. [2017] fornece um conjunto de dados denominado *Numenta*, uma métrica denominada NAB *Score* e experimentos para *benchmark* de comparação de métodos de detecção de eventos *online*. Diversos métodos foram comparados na detecção *online* neste conjunto de dados e os três métodos com informações publicadas com melhores desempenhos foram MTH,

Relative Entropy e *Twitter ADVec* [Ahmad et al., 2017]. Os métodos foram comparados usando as métricas NAB *Score* e tempo de latência em micro segundos (ms).

A métrica NAB *Score* parte de cálculos das métricas de classificação VP, VN, FP e FN, mas inclui uma forma de recompensar detecções precoces e penalizar detecções tardias. Para isso é calculada uma janela temporal ao redor do ponto no tempo onde o rótulo de um conjunto de dados indica a existência de um evento. Nesta abordagem os verdadeiros e falsos positivos não são calculados pelo acerto ou erro exato de um ponto no tempo como na matriz de confusão, mas pela indicação de um evento dentro da janela calculada na métrica. Além disso, detecções logo no início da janela tem maior peso [Ahmad et al., 2017].

Na prática quando se trabalha com detecção de eventos *online* em séries em *streaming* não há rótulos disponíveis, uma vez que cada observação ou conjunto de observações novas será analisada em fluxo contínuo. Contudo, como pode ser visto na Tabela III.1, para avaliar e comparar métodos é normal o uso de métricas comuns em tarefas de classificação que dependem de rótulos para seu cálculo, como a matriz de confusão, acurácia, precisão, revocação e F_1 que são obtidas pela comparação dos eventos reais (rotulados) com aqueles detectados pelo método avaliado [Han et al., 2011; Escobar et al., 2021; Ahmad et al., 2017]. As métricas relacionadas ao desempenho computacional do método, como tempo de execução, latência, consumo de memória e CPU, são mais simples de analisar por não dependerem de rótulos, podendo ser coletadas e comparadas apenas para identificar os métodos com maior ou menor valores [Hasani, 2020; Ahmad et al., 2017].

Tabela III.1: Trabalhos de comparação de métodos: Conjuntos de dados e métricas usadas

Trabalho	Conjuntos de Dados	Métricas
Hasani [2020]	*Numenta Benchmark*	Tempo de Execução, Uso de CPU
Salles et al. [2020]	*Numenta Benchmark*, *Gecco Challenge*, *Yahoo Labs*	F_1-*Score*, matriz de confusão, acurácia
Ren et al. [2019]	KPI, *Yahoo Labs*, Microsoft	F_1-*Score*, precisão, revocação
Ahmad et al. [2017]	*Numenta Benchmark*	NAB *Score*, latência

Uma análise de métodos com foco na avaliação de viés temporal é apresentada por Escobar et al. [2021]. O viés temporal está relacionado à identificação da tendência de métodos de antecipar ou atrasar as detecções de eventos. O trabalho discute que métricas tradicionais baseadas no acerto dos pontos exatos dos eventos são insuficientes para este tipo de análise. Além disso, é levantada a questão que para algumas aplicações pode ser desejável que a detecção seja antecipada ao invés de ocorrer apenas em um ponto exato da série [Singh and Olinsky, 2017; Escobar, 2020].

Além dos conjuntos de dados sintetizados na Tabela III.1, um conjunto de dados denominado *dataset for cloud-native memory anomalies* (RARE) é apresentado sobre monitoramento de uso de

memória em serviços de nuvem [Lomio et al., 2020]. O objetivo da publicação do RARE não foi a avaliação de métodos, mas suportar outros pesquisadores a realizarem esta tarefa com provimento de dados. Por isso, sua comparação foi com conjuntos de dados *Yahoo Labs*[3] e *Numenta Benchmark* [Ahmad et al., 2017][4] que contém dados sintéticos e reais sobre monitoramento de serviços de nuvem com rótulos de eventos. Outro conjunto de dados normalmente usado para avaliar métodos de detecção de eventos é o *Gecco Challenge*[5] que também contém dados rotulados sobre qualidade da água [Moritz et al., 2018].

Confrontando os resultados de muitos trabalhos sobre detecção de eventos, Wu and Keogh [2021] afirmam que conjuntos de dados amplamente usados nesta área como *Numenta* e *Yahoo* apresentam falhas graves que geram resultados não confiáveis e ilusão de progresso. As falhas são classificadas em: (i) trivialidade, (ii) densidade irreal de anomalias, (iii) área verdadeira (verdadeiros positivos e verdadeiros negativos) rotulada erroneamente e (iv) viés execução até de falha. Para lidar com estas falhas é apresentado o conjunto de dados *UCR Anomaly Archire* que evita as falhas citadas, composto de diversas séries com eventos rotulados [Wu and Keogh, 2021].

Ao comparar métodos, além dos aspectos já citados, é importante analisar requerimentos desejáveis para detecção de eventos em tempo real [Stahmann and Rieger, 2021]. No geral são exemplos de requerimentos desejáveis tanto dos métodos quanto dos dados e da forma de execução: a origem e composição adequada dos dados, capacidade de resposta, escalabilidade, baixa latência, tolerância a falhas e pós-processamento (avaliação, notificação e ajustes) [Stahmann and Rieger, 2021; Vyas et al., 2021]. São requerimentos específicos dos métodos de detecção: capacidade de aprendizado e adaptação contínua, lidar com séries não estacionárias e mudanças em geral ao longo do tempo, detecção de eventos tão cedo quanto possível, minimizar falsos positivos e falsos negativos [Ahmad et al., 2017].

III.3 Síntese de Trabalhos Relacionados

Uma visão dos trabalhos relacionados, conforme análise das Seções anteriores deste Capítulo, é sintetizada nesta Seção para reforçar pontos relevantes observados, lacunas e aspectos relacionadas à evolução da pesquisa. A Figura III.1 ilustra a distribuição ao longo dos anos dos 379 documentos obtidos a partir da *string* de busca mencionada no início deste Capítulo. Como se percebe a produção acadêmica sobre detecção *online* de eventos em séries temporais apresenta um crescimento expressivo nos últimos anos, especialmente a partir de 2011. Este crescimento reforça a importância da pesquisa sobre detecção *online* de eventos em séries temporais.

A classificação da detecção baseada em **predição** e em **fronteiras** é uma simplificação que

[3]Disponível em https://yahooresearch.tumblr.com/post/114590420346/
[4]Disponível no pacote R OTSAD
[5]Disponível no pacote R EventDetectR

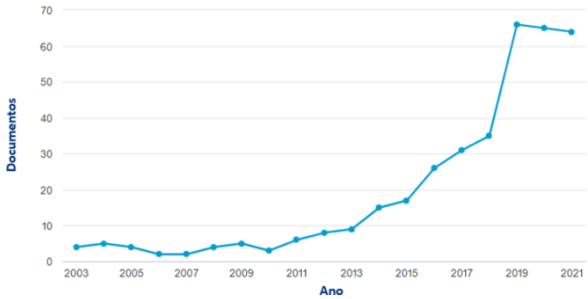

Figura III.1: Evolução da quantidade de publicações: Quantidade de trabalhos sobre detecção de *online* publicados por ano até 2021. Fonte: Análise do resultado da *string* de busca na base *Scopus*

não exaure o funcionamento dos métodos. Há inclusive métodos que não se adéquam perfeitamente a esta classificação como o MF-stacked LSTM-EWMA [Zhang et al., 2020] que usa ambas atividades (predição e fronteiras) ou o método proposto por Rettig et al. [2015] que não as usa. Entretanto, essa classificação é relevante para planejamento adequado de experimentos de comparação de métodos. Esta classificação pode indicar, por exemplo, a necessidade de uso de dados prévios antes de iniciar detecções em novas observações do *streaming*, seja para treinar o modelo preditivo ou para estabelecer os limites da fronteira.

É importante na comparação dos métodos a escolha adequada de uma linha de base que identifique o desempenho mínimo de um método e o estado da arte a ser superado, bem como a maior diversidade possível de tipos de séries. Estes aspectos serão tratados em maiores detalhes na avaliação experimental. Contudo, desta síntese de trabalhos relacionados, é possível intuir a necessidade de comparação com os métodos mais recentes com resultados iguais ou melhores ao estado da arte como MAD [Ren et al., 2019] e MF-*stacked* LSTM-EWMA [Zhang et al., 2020] ou métodos que já funcionem adequadamente mesmo na detecção *offline* [Carmona, 2014; Hasani, 2017; Dancho and Vaughan, 2020; Guralnik and Srivastava, 1999; Takeuchi and Yamanishi, 2006; Escobar et al., 2021].

Quanto aos trabalhos que comparam métodos de detecção *online* foram percebidos como relevantes os três aspectos a seguir: (i) forma de execução dos métodos, (ii) conjuntos de dados usados e (iii) métricas para comparação dos métodos. A forma de execução dos métodos, aspecto (i), foi discutida na Seção III.1. Os trabalhos de comparação dos métodos, por sua vez, foram abordados na Seção III.2 que aprofundou os aspectos (ii) e (iii), conjuntos de dados e as métricas usadas cuja Tabela III.1 exibe uma síntese.

Percebe-se uma concentração no uso alguns conjuntos de dados como *Numenta Benchmark* e *Yahoo Labs*, conforme Tabela III.1. Esta concentração por um lado reforça a importância destas séries, mas pode gerar fragilidades como excesso de especialização de métodos e risco de baixa capacidade de generalização. Por isso, são importantes as discussões que levaram a proposição do RARE [Lomio et al., 2020] e *UCR Anomaly Archive* [Wu and Keogh, 2021] que complementam ou

contestam, respectivamente, as séries de uso predominante para avaliação de métodos.

Algumas recomendações de Wu and Keogh [2021] são severas, como abandonar os conjuntos de dados mais usados na área de pesquisa e rever quaisquer publicações que os tenha como principal base de dados. A falha (iii) área verdadeira rotulada erroneamente, por exemplo, é baseada na análise visual das regiões das séries com anomalias e na incoerência dos rótulos contidos nas séries. Esta técnica (análise visual) é importante para explorar a série, mas não garante que todas as séries refutadas por Wu and Keogh [2021] repitam os problemas dos intervalos analisados. De qualquer modo, mesmo que não se desconsidere totalmente o uso de séries como *Numenta* e *Yahoo Labs*, fica evidente a importância da seleção adequada de múltiplos conjuntos de dados para avaliação de métodos de detecção eventos.

O uso de lotes para processamento de dados tem aplicações tanto para tarefas de processamento *offline* quanto *online*. Normalmente há uma associação direta de processamento *offline* com o termo processamento em lote, mas esta associação é mais adequada quando se refere ao acesso prévio aos dados completos [Truong et al., 2020]. Contudo, como foi visto o processamento de dados em *streaming* pode ocorrer através uso de pequenos lotes [Rettig et al., 2015; Hiraman et al., 2018].

Esta abordagem de pequenos lotes é similar ao conceito de subsequências ou janelas deslizantes [Fu, 2011; Chiu et al., 2003; Lin and Keogh, 2004]. A divisão de séries temporais em lotes foi experimentada para o processamento em *streaming* [Iwashita and Papa, 2019]. Contudo Iwashita and Papa [2019] não aplica sua abordagem à detecção de eventos, mas à identificação de mudanças de conceito.

Os trabalhos analisados neste Capítulo apresentaram diversidade de métodos para detecção de eventos *online*, aplicação de métricas para comparação dos métodos, além de conjuntos de dados e ferramentas para processamento de séries em *streaming*. Contudo, segundo análise dos trabalhos relacionados e o melhor conhecimento dos autores, ainda há carência de trabalhos focados especificamente na comparação de métodos de detecção de eventos em dados em *streaming*. Especialmente, nota-se ausência de comparações que sejam independente de solução específica de processamento de *streaming* e usem lotes deslizantes de diferentes tamanhos.

Diante das considerações precedentes, a avaliação proposta neste trabalho visa explorar esta lacuna na detecção de eventos *online* em séries temporais em *streaming*, conforme abordado no Capítulo IV.

Capítulo IV Metodologia

Como resultados preliminares deste trabalho, algumas contribuições para o estudo da detecção de eventos alcançadas no âmbito do mestrado já foram publicadas em artigos [Ogasawara et al., 2021; Gea et al., 2021; Escobar et al., 2021]. Uma destas contribuições deu origem à taxonomia apresentada no Capítulo II que explora diversos aspectos da detecção *online* de eventos em séries em *streaming* [Ogasawara et al., 2021]. Os cuidados na transformação de dados, pela remoção de anomalias em séries financeiras e seus efeitos na perda de informações foi outra contribuição relevante [Gea et al., 2021]. Também foram explorados os resultados de diversos métodos para avaliar aspectos relacionados ao viés temporal, atrasos e antecipação nas detecções [Escobar et al., 2021].

Como outro resultado preliminar, um artigo foi submetido e aceito para a *International Joint Conference on Neural Networks*(IJCNN) com a publicação de um novo método denominado *Forward and Backward Inertial Anomaly Detector* (FBI-AD). Este método identifica pontos de mudança e anomalias de tendência e superou a acurácia e tempo de execução de outros métodos. Os resultados do FBI-AD foram promissores mesmo quando ele foi comparado com métodos do estado da arte como MAD, mas os experimentos foram realizados em detecções *offline*.

Apesar destes resultados preliminares serem promissores, os experimentos de comparação de métodos foram realizados em cenário *offline*. Por isso, surgem questões adicionais como: os resultados de métodos avaliados se mantém no cenário *online*? Métodos que realizam etapas de transformação de dados conseguem melhor adaptação às mudanças esperadas em séries em *streaming*? Métodos que são melhores em situações ou séries específicas continuam válidos em séries em *streaming*? Como o processamento das séries em *streaming* impactam o atraso nas detecções e o tempo de execução dos métodos?

Para explorar estas questões e contribuir para o avanço na área de detecção de eventos *online*, o presente trabalho propõe uma análise do uso de lotes deslizantes na detecção de eventos em séries temporais em *streaming*. Desta forma, a pesquisa contribuirá para testar diretamente a hipótese (3), proposta no Capítulo I, relacionada ao uso de lotes deslizantes que consigam lidar com subsequências menores da série poderia resultar em detecção precoce de eventos e redução do custo computacional do processamento *online*, mas sem os impactos negativos da detecção tardia do processamento *offline*.

O *workflow* para detecção de eventos em *streaming*, que compõe a metodologia deste trabalho, é apresentado na Figura IV.1. A análise proposta, contribui para a capacidade de experimentação de métodos de detecção de eventos em dados em *streaming*. Adicionalmente, contribuirá para (i) celeridade na capacidade de resposta aos eventos, pela maior proximidade do ambiente onde dados são gerados proporcionado pela abordagem baseada em *streaming*; (ii) comparação de diferentes métodos de detecção *online* e identificação de seu desempenho quanto à capacidade de identificar eventos reais e quanto ao tempo de execução.

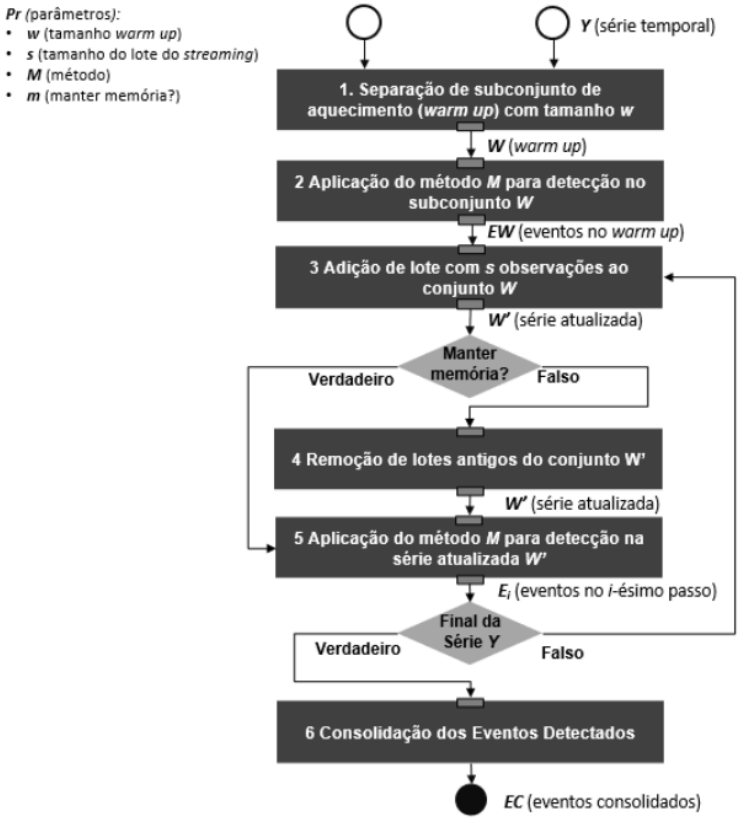

Figura IV.1: Diagrama do *workflow* proposto: Aplicação de métodos de detecção de eventos em séries em *streaming*

O *workflow* de execução dos métodos, apresentado na Figura IV.1, recebe uma série temporal Y e parâmetros adicionais para a avaliação de métodos Pr. Os parâmetros (Pr) se referem ao método a ser executado e as configurações da execução do *workflow*. Estes parâmetros adicionais de configuração são: w - um valor inteiro múltiplo do tamanho de lote (s), usado para definir o tamanho do **subconjunto para aquecimento (*warm up*)**. Este subconjunto se refere à quantidade mínima de observações para início das detecções; s - **tamanho do lote** em cada iteração da simulação do *streaming*; M - o **método que será aplicado** para detecção com seus respectivos hiperparâmetros; m - Valor lógico para definir se o *workflow* manterá a **memória dos lotes**, ou seja, como deve percorrer a série e gerar a versão atualizada (W'): **(i) Verdadeiro**: manter os lotes antigos; **(ii) Falso**: remover o último lote à cada iteração.

A análise é realizada através da aplicação do *workflow* para execução dos métodos que serão selecionados na avaliação experimental. Os conjuntos de dados para análise, compostos por séries e eventos rotulados também são definidos na avaliação experimental. Para cada série temporal analisada por um determinado método devem ser armazenados os resultados das detecções para posterior comparação com os rótulos existentes, bem como monitorados os tempos de execução para comparação.

A série Y é usada para simular um *streaming* de dados e sua submissão à detecção de eventos com o método avaliado. O processo começa pela separação de uma quantidade de observações iniciais, denominada *warm up*, para permitir que o método avaliado tenha quantidade mínima de dados para iniciar as detações. Para simular o processamento da série em *streaming*, o *workflow* prossegue, percorrendo a série em pequenos lotes de maneira iterativa até que toda a série seja percorrida. Uma intuição deste processo de percorrer a série através dos lotes é ilustrado nas Figuras IV.2 e IV.3 com ou sem manutenção da memória dos dados de lotes antigos, respectivamente.

Para realizar a detecção *online* de eventos, o lote mais recente sempre é o objeto principal das detecções. Como saída final é obtido um vetor EC com os eventos consolidados. A análise da saída EC permite avaliar o resultado das detecções à medida que os lotes foram submetidos à análise ao longo do tempo.

Os passos do $workflow$ proposto, cuja visão geral da lógica está na Figura IV.1, estão detalhados a seguir: (i) **separação do *warm up***; (ii) **aplicação do método para detecção no subconjunto de aquecimento**; (iii) **simulação do *streaming***; (iv) **consolidação dos eventos detectados**.

Em (i) **separação do *warm up*** tem-se a separação do *warm up*, identificado como W, com tamanho w é realizada no passo 1 da Figura IV.1. Este subconjunto é uma subsequência da série Y, composta da observação y_1 até y_w, onde y_1 é a observação mais antiga da série Y e y_w é a observação na w-ésima posição da série. Aqui w é um valor inteiro que determina a quantidade mínima de observações para início das detecções. O valor de w é arbitrário, mas deve ser múltiplo de s para simplificação da tarefa de percorrer a série em lotes, mantendo a quantidade mínima de dados para comparação com as novas observações do lote ativo e gerenciamento da memória dos lotes antigos.

Em seguida, tem-se (ii) **aplicação do método para detecção no subconjunto de aquecimento**. Este passo consiste em aplicar o método M, com os hiperparâmetros fornecidos, para detecção de eventos no subconjunto W (*warm up*). Como resultado deste passo é gerada a lista de eventos EW detectados no *warm up*.

Em (iii) **simulação do *streaming***, correspondente aos passos 3 a 5 da Figura IV.1, simula-se o *streaming* de dados em uma série e a detecção de eventos nesta série à medida que novas observações

são recebidas. Para isso é feita a adição de s observações ao conjunto W, simulando a chegada de novas observações em uma série em *streaming*. Esta tarefa tem como saída um novo subconjunto W' da série original Y. Antes da detecção de eventos no *streaming*, se o parâmetro lógico m estiver definido como **verdadeiro** o passo condicional 4 não é executado o que mantém a memória dos lotes antigos. Contudo, se o parâmetro estiver definido como **falso** é executado, removendo o lote mais antigo do subconjunto W'. Este passo funciona como uma forma de manter ou remover a memória de observações antigas da série no *streaming*.

Em seguida é feito uso da série atualizada W' para aplicação do método M. Neste passo 5 é construído o modelo nos lotes completos e executada a detecção no lote ativo (mais recente). O método em execução avalia cada nova observação nova no lote ativo para detectar se deverá ser marcada como um evento.

Em (iv) **consolidação dos eventos detectados** os eventos E_i, detectados durante as sucessivas rodadas da simulação do *streaming* de dados, são consolidados em EC. Os eventos consolidados (EC) contém o resultado das detecções de cada passo do *streaming*, E_i, para identificar tanto os eventos quanto o momento em que foi realizada a detecção.

A avaliação proposta neste trabalho usará o *workflow* descrito (Figura IV.1) para analisar a influência do uso de lotes deslizantes nos resultados da detecção de eventos em séries temporais em *streaming*. Para isso, serão realizados experimentos durante a elaboração da dissertação para testar diferentes configurações de lotes como: alteração dos tamanhos, manutenção ou não da memória de dados antigos e comparação dos resultados em séries distintas. A abordagem proposta não é dependente do método a ser executado ou de plataforma de *streaming* específica.

A diferença entre a manutenção da memória dos dados completos da série para análise ou apenas dos últimos lotes é ilustrada nas Figuras IV.2 e IV.2. Na execução do *workflow* mantendo a memória os lotes são adicionados e a série é analisada de maneira cumulativa. Desta forma há mais informações para uso pelos métodos que pode ajudar a aumentar a acurácia. Contudo, é natural que ocorra um aumento gradativo do custo computacional e eventuais falhas nas detecções pela influência de observações antigas nas detecções.

Quando o *workflow* é executado sem manutenção de memória os lotes antigos são removidos (esquecidos) à medida que novos lotes são adicionados (Figura IV.2). Espera-se avaliar nos experimentos se esta opção gera o efeito inverso à preservação da memória, ou seja, menor custo computacional, maior influência das observações recentes na detecção de eventos e seus efeitos na detecção precoce. Como o tamanho do *warm up* (w) é múltiplo do tamanho de lotes (s), é possível esquecer lotes antigos e manter a quantidade mínima de dados para o funcionamento do método em execução, conforme descrito no passo 4 da Figura IV.1.

Tendo em vista a independência de métodos ou plataformas de *streaming* específicos, será

Figura IV.2: Ilustração da detecção em lotes deslizantes 1: Adição de um novo lote a cada iteração, simulando o *streaming* com memória completa dos dados

Figura IV.3: Ilustração da detecção em lotes deslizantes 2: Adição de um novo lote a cada iteração, simulando o *streaming* esquecendo os lotes mais antigos

possível a experimentação e comparação de métodos distintos de maneira flexível. A implementação do *workflow* proposto na Figura IV.1 será melhor explorada na avaliação experimental durante o desenvolvimento da dissertação e poderá ser reproduziada através do código disponível[1]. A metodologia apresentada neste Capítulo também será detalhada de maneira mais aprofundada durante a elaboração da dissertação.

[1]Implementação preliminar do *workflow* de experimentação de métodos na linguagem R disponível em: https://github.com/cefet-rj-dal/jlima/tree/main/harbinger/src/cefet/harbinger/R

Capítulo V Considerações Parciais

Como foi discutido no Capítulo II a produção acadêmica sobre detecção *online* apresentou expressivo crescimento nos últimos anos, especialmente na última década. Esse crescimento na pesquisa vai ao encontro da importância do tema nas organizações, marcado pela onipresença da internet e do avanço de dispositivos IoT, plataformas de nuvem e geração de dados em *streaming* em geral. A detecção de eventos *online* é importante neste contexto porque permite o monitoramento das séries temporais em *streaming* para identificação tempestiva de situações indesejadas. Estes eventos são observações em uma série temporal com significado especial para a área de domínio, tais como fraudes, mudanças inesperadas em séries financeiras, anomalias em dados médicos, monitoramento de sensores em equipamentos usados na exploração de petróleo.

Os trabalhos relacionados analisados apresentaram diversidade de métodos para detecção de eventos *online*, aplicação de métricas para comparação dos métodos, além de conjuntos de dados e ferramentas para processamento de séries em *streaming*. Contudo, ainda há carência de trabalhos focados especificamente na comparação de métodos de detecção de eventos em dados em *streaming* que sejam independente de solução específica de processamento de *streaming* e usando lotes deslizantes de diferentes tamanhos. Por isso, a avaliação proposta neste trabalho visa explorar esta lacuna na detecção de eventos *online* em séries temporais em *streaming*, conforme metodologia proposta no Capítulo IV.

O presente trabalho propôs uma análise do uso de lotes deslizantes na detecção de eventos em séries temporais em *streaming*. Desta forma, a pesquisa contribuirá para testar diretamente a hipótese (3), proposta no Capítulo I, relacionada ao uso de lotes deslizantes que consigam lidar com subsequências menores da série poderia resultar em detecção precoce de eventos e redução do custo computacional do processamento *online*, mas sem os impactos negativos da detecção tardia do processamento *offline*.

Além disso, a realização de uma avaliação experimental mais abrangente durante o desenvolvimento da dissertação contribuirá para: (i) avaliar o impacto na velocidade da capacidade de resposta aos eventos pela experimentação de diferentes configurações de lotes deslizantes em séries em *streaming* (tamanho de cada lote e memória dos dados); (ii) comparação de diferentes métodos de detecção *online* quanto à capacidade de detectar eventos corretamente e ao tempo de execução.

Apesar da abrangência da exploração do tema, há limitações no estágio atual da pesquisa que

serão tratadas no desenvolvimento da dissertação ou em trabalhos futuros. O aprofundamento da análise dos trabalhos relacionados, por exemplo, poderão indicar aspectos adicionais dos métodos para considerar na avaliação experimental. A metodologia foi abordada no Capítulo IV com base nos estudos já empreendidos e testes preliminares simplificados[1], mas deverá ser mais detalhada no texto final.

Os testes preliminares simplificados também serão aprofundados no texto final da dissertação. Parte dos testes em questão foram baseados em métodos e conjuntos de dados avaliados e tratados em artigos já publicados no âmbito do mestrado [Ogasawara et al., 2021; Gea et al., 2021; Escobar et al., 2021] ou em processo de publicação. Conforme discutido no início do Capítulo IV, alguns destes resultados preliminares foram obtidos em cenário de detecção *offline*. Contudo, contribuíram para levantamento de questões abordadas ao longo deste trabalho e serão uma base importante para os experimentos usando a metodologia proposta no cenário específico de detecção *online* de eventos em séries em *streaming*.

Por fim, através do aproveitamento dos resultados e experimentos a serem produzidos nesta pequisa, vislumbra-se abertura de caminhos para trabalhos futuros para experimentação e combinação de métodos de detecção de eventos em dados em *streaming*. Por exemplo a mudança do método aplicado quando a série muda de comportamento ao longo do tempo para maior capacidade de adaptação das detecções e a combinação de mais de um método para melhoria na acurácia das detecções.

[1]Disponível em https://github.com/cefet-rj-dal/jlima/tree/main/harbinger/src/cefet/harbinger/R

Referências Bibliográficas

L. Atzori, A. Iera, and G. Morabito. The Internet of Things: A survey. *Computer Networks*, 54 (15):2787–2805, 2010. 1

R.A. Ariyaluran Habeeb, F. Nasaruddin, A. Gani, I.A. Targio Hashem, E. Ahmed, and M. Imran. Real-time big data processing for anomaly detection: A Survey. *International Journal of Information Management*, 45:289–307, 2019. 1, 2, 10, 11, 12, 13, 15

P. Esling and C. Agon. Time-series data mining. *ACM Computing Surveys*, 45(1), 2012. 1, 4, 9

V. Chandola, A. Banerjee, and V. Kumar. Anomaly detection: A survey. *ACM Computing Surveys*, 41(3), 2009. 1, 10, 11, 16

P.D. Talagala, R.J. Hyndman, K. Smith-Miles, S. Kandanaarachchi, and M.A. Muñoz. Anomaly Detection in Streaming Nonstationary Temporal Data. *Journal of Computational and Graphical Statistics*, 29(1):13–27, 2020. 1, 2, 6, 12, 13, 15, 17

C. Truong, L. Oudre, and N. Vayatis. Selective review of offline change point detection methods. *Signal Processing*, 167, 2020. 1, 10, 12, 16, 22

Subutai Ahmad, Alexander Lavin, Scott Purdy, and Zuha Agha. Unsupervised real-time anomaly detection for streaming data. *Neurocomputing*, 262:134–147, 2017. ISSN 0925-2312. 1, 13, 16, 18, 19, 20

F. Lomio, D. M. Baselga, S. Moreschini, H. Huttunen, and D. Taibi. Rare: A labeled dataset for cloud-native memory anomalies. In *MaLTeSQuE 2020 - Proceedings of the 4th ACM SIGSOFT International Workshop on Machine-Learning Techniques for Software-Quality Evaluation, Co-located with ESEC/FSE 2020*, pages 19–24, 2020. 1, 20, 21

Luciana Escobar, Rebecca Salles, Janio Lima, Cristiane Gea, Lais Baroni, Artur Ziviani, Paulo Pires, Flavia Delicato, Rafaelli Coutinho, Laura Assis, and Eduardo Ogasawara. Evaluating temporal bias in time series event detection methods. *Journal of Information and Data Management*, 12(3), 2021. 1, 16, 19, 21, 23, 29

R. Wu and E. Keogh. Current time series anomaly detection benchmarks are flawed and are creating the illusion of progress. *IEEE Transactions on Knowledge and Data Engineering*, 2021. 1, 20, 21, 22

L. Rettig, M. Khayati, P. Cudre-Mauroux, and M. Piorkowski. Online anomaly detection over big data streams. In *Proceedings - 2015 IEEE International Conference on Big Data, IEEE Big Data 2015*, pages 1113–1122, 2015. 2, 12, 13, 16, 21, 22

B. R. Hiraman, M. C. Viresh, and C. K. Abhijeet. A study of apache kafka in big data stream processing. In *2018 International Conference on Information, Communication, Engineering and Technology, ICICET 2018*, 2018. 2, 12, 22

A. S. Iwashita and J. P. Papa. An overview on concept drift learning. *IEEE Access*, 7:1532–1547, 2019. 2, 6, 13, 22

Rebecca Salles, Luciana Escobar, Lais Baroni, Roccio Zorrilla, Artur Ziviani, Vincius Kreischer, Flavia Delicato, Paulo F. Pires, Luciano Maia, Rafaelli Coutinho, Laura Assis, and Eduardo Ogasawara. Harbinger: Um framework para integração e análise de métodos de detecção de eventos em séries temporais. In *Anais do Simpósio Brasileiro de Banco de Dados (SBBD)*, pages 73–84. SBC, September 2020. 2, 13, 16, 18, 19

R. H. Shumway and D. S. Stoffer. *Time Series Analysis and Its Applications: With R Examples*. Springer, New York, NY, 4 edition, April 2017. ISBN 978-3-319-52451-1. 4, 5, 7, 17

Dominique M. Hanssens, Leonard J. Parsons, and Randall L. Schultz. *Market Response Models: Econometric and Time Series Analysis*. Springer, Boston, Mass., 2nd edition edition, January 2003. ISBN 978-1-4020-7368-7. 4

Eduardo Ogasawara, Rebecca Salles, Luciana Escobar, Lais Baroni, Janio Lima, and Fabio Porto. Online event detection for sensor data. In *Proceedings of the Ibero-Latin-American Congress on Computational Methods in Engineering*, 2021. 4, 6, 10, 11, 15, 23, 29

Rebecca Salles, Kele Belloze, Fabio Porto, Pedro H Gonzalez, and Eduardo Ogasawara. Nonstationary time series transformation methods: An experimental review. *Knowledge-Based Systems*, 164:274–291, 2019. ISSN 0950-7051. doi: https://doi.org/10.1016/j.knosys.2018.10.041. 4, 16

J. Demšar and Z. Bosnić. Detecting concept drift in data streams using model explanation. *Expert Systems with Applications*, 92:546–559, 2018. 6

Indrė Žliobaitė. Learning under concept drift: an overview, 2010. URL `https://arxiv.org/abs/1010.4784`. 6

Jiawei Han, Micheline Kamber, and Jian Pei. *Data Mining: Concepts and Techniques*. Morgan Kaufmann, Haryana, India; Burlington, MA, 3 edition, July 2011. ISBN 978-0-12-381479-1. 6, 7, 8, 9, 11, 13, 14, 19

C. McLaren, D. Elliott, and R. Kirchner. *Handbook on Seasonal Adjustment*. Eurostat, Luxembourg, European Union, 1nd edition edition, 05 2018. ISBN 978-92-79-80170-9. 7

E. Ogasawara, L.C. Martinez, D. De Oliveira, G. Zimbrão, G.L. Pappa, and M. Mattoso. Adaptive Normalization: A novel data normalization approach for non-stationary time series. In *Proceedings of the International Joint Conference on Neural Networks*, 2010. 7, 8, 9

J. Lin, E. Keogh, and W. Truppel. Clustering of streaming time series is meaningless. In *Proceedings of the 8th ACM SIGMOD Workshop on Research Issues in Data Mining and Knowledge Discovery, DMKD '03*, pages 56–65, 2003. 7, 9, 16

B. Chiu, E. Keogh, and S. Lonardi. Probabilistc discovery of time series motifs. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 493–498, 2003. 7, 9, 16, 22

Luciana Escobar. Análise comparativa de métodos para detecção de eventos em séries temporais. Master's thesis, Centro Federal de Educação Tecnológica Celso Suckow da Fonseca (CEFET/RJ), 2020. 7, 19

R. Marik and L. Bohac. Non-stationary events detection based on extrema value theory. In *2018 5th International Conference on Systems and Informatics, ICSAI 2018*, pages 889–894, 2019. 8, 10

Tak-Chung Fu. A review on time series data mining. *Engineering Applications of Artificial Intelligence*, 24(1):164–181, 2011. 9, 22

Jessica Lin and Eamonn Keogh. Finding or not finding rules in time series. In Jane M Binner, Graham Kendall, and Shu-Heng Chen, editors, *Applications of Artificial Intelligence in Finance and Economics*, volume 19 of *Advances in Econometrics*, pages 175–201. Emerald Group Publishing Limited, 2004. 9, 22

A. Gensler and B. Sick. Performing event detection in time series with swiftevent: an algorithm with supervised learning of detection criteria. *Pattern Analysis and Applications*, 21(2):543–562, 2018. 10

J.-I. Takeuchi and K. Yamanishi. A unifying framework for detecting outliers and change points from time series. *IEEE Transactions on Knowledge and Data Engineering*, 18(4):482–492, 2006. 11, 16, 21

Kevin M. Carter and William W. Streilein. Probabilistic reasoning for streaming anomaly detection. In *2012 IEEE Statistical Signal Processing Workshop (SSP)*, pages 377–380, 2012. doi: 10.1109/SSP.2012.6319708. 12, 13

H. Ren, B. Xu, Y. Wang, C. Yi, C. Huang, X. Kou, T. Xing, M. Yang, J. Tong, and Q. Zhang. Time-series anomaly detection service at Microsoft. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 3009–3017, 2019. 13, 17, 19, 21

Z. Hasani. Anomaly detection algorithms for streaming data: Performance comparison. *Journal of Computer Science*, 16(7):950–955, 2020. 13, 18, 19

R. Carmona. *Statistical Analysis of Financial Data in R.* Springer-Verlag New York, 2014. 16, 21

Z. Hasani. Robust anomaly detection algorithms for real-time big data: Comparison of algorithms. In *2017 6th Mediterranean Conference on Embedded Computing, MECO 2017 - Including ECYPS 2017, Proceedings*, 2017. 16, 21

Matt Dancho and Davis Vaughan. *Anomalize: Tidy Anomaly Detection*, 2020. URL `https://CRAN.R-project.org/package=anomalize`. 16, 21

Valery Guralnik and Jaideep Srivastava. Event Detection from Time Series Data. In *Proceedings of the Fifth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '99, pages 33–42, New York, NY, USA, 1999. ACM. ISBN 978-1-58113-143-7. doi: 10.1145/312129.312190. event-place: San Diego, California, USA. 16, 21

Vladislav Ishimtsev, Alexander Bernstein, Evgeny Burnaev, and Ivan Nazarov. Conformal k-NN anomaly detector for univariate data streams. In *Proceedings of the Sixth Workshop on Conformal and Probabilistic Prediction and Applications*, volume 60 of *Proceedings of Machine Learning Research*, pages 213–227. PMLR, 13–16 Jun 2017. 16

Rikard Laxhammar. *Conformal anomaly detection: Detecting abnormal trajectories in surveillance applications.* PhD thesis, University of Skovde, 2014. 16

M. Munir, S. A. Siddiqui, M. A. Chattha, A. Dengel, and S. Ahmed. Fusead: Unsupervised anomaly detection in streaming sensors data by fusing statistical and deep learning models. *Sensors (Switzerland)*, 19(11), 2019. 17

M. Zhang, J. Guo, X. Li, and R. Jin. Data-driven anomaly detection approach for time-series streaming data. *Sensors (Switzerland)*, 20(19):1–17, 2020. 17, 21

N. Singh and C. Olinsky. Demystifying numenta anomaly benchmark. In *Proceedings of the International Joint Conference on Neural Networks*, pages 1570–1577. IEEE, 2017. 19

Steffen Moritz, Frederik Rehbach, Sowmya Chandrasekaran, Margarita Rebolledo, and Thomas Bartz-Beielstein. GECCO Industrial Challenge 2018 Dataset: A water quality dataset for the 'Internet of Things: Online Anomaly Detection for Drinking Water Quality' competition at the Genetic and Evolutionary Computation Conference 2018, Kyoto, Japan. Zenodo, 2018. 20

Philip Stahmann and Bodo Rieger. Requirements identification for real-time anomaly detection in industrie 4.0 machine groups: A structured literature review. In *Proceedings of the 54th Hawaii International Conference on System Sciences*, 2021. doi: https://doi.org/10.24251/HICSS.2021.696. 20

S. Vyas, R. K. Tyagi, C. Jain, and S. Sahu. Literature review: A comparative study of real time streaming technologies and apache kafka. In *Proceedings - 2021 4th International Conference on Computational Intelligence and Communication Technologies, CCICT 2021*, pages 146–153, 2021. 20

C. Gea, J. Lima, Eduardo Bezerra, and Eduardo Ogasawara. Análise de métodos de tratamento de outliers para predição dos retornos de índices de ações negociados em bolsa. In *Anais do Simpósio Brasileiro de Banco de Dados (SBBD)*. SBC, 2021. 23, 29